

Fiche 304 — Facteurs : classification discrète, `tapply()` et tableaux irréguliers

Matière	Code · Langage R
Cours source	R Core Team, <i>An Introduction to R</i> 4.6.1 — chapitre 4 « Ordered and unordered factors » (§4.1 un exemple, §4.2 <code>tapply()</code> et tableaux irréguliers, §4.3 facteurs ordonnés)
Sources d'appoint	<i>R Language Definition</i> 4.6.1, §2.3.1 « Factors » ; §3.4.1 « Indexing by vectors » (le piège de l'index facteur)
Difficulté	Intermédiaire — court chapitre, mais deux pièges majeurs
Temps d'étude estimé	1 h
Prérequis	Fiches 301 à 303 (vecteurs, indexation, attributs)
Concepts clés	classification discrète, <code>factor()</code> , attribut <code>levels</code> , classe <code>"factor"</code> , ordre alphabétique des niveaux, <code>levels()</code> , affichage particulier, <code>tapply()</code> , coercion automatique par <code>as.factor()</code> , tableau irrégulier (<i>ragged array</i>), <code>ordered()</code> , classe <code>c("ordered", "factor")</code> , <code>contrasts</code> , implémentation par entiers
À retenir en priorité	Les niveaux sont alphabétiques par défaut · <code>tapply()</code> = « appliquer par groupe » · <code>sd()</code> n'est pas l'erreur type · ne jamais s'appuyer sur le codage entier.

Vue d'ensemble

UN FACTEUR	un vecteur qui spécifie une CLASSIFICATION DISCRETE (un groupage) des composantes d'autres vecteurs DE MEME LONGUEUR
CE QU'IL EST	un vecteur d'ENTIERS + attribut <code>levels</code> + class <code>"factor"</code> (mais NE PAS s'appuyer la-dessus : detail d'implementation)
CREER	<code>statef <- factor(state)</code>
LES NIVEAUX	<code>levels(statef)</code> -> ordre ALPHABETIQUE par défaut ou celui donne explicitement a <code>factor()</code>

L'OUTIL	<code>tapply(donnees, facteur, fonction)</code> « appliquer la fonction a CHAQUE GROUPE defini par les niveaux » resultat : un vecteur de la LONGUEUR DE levels, etiquete par eux
TABLEAU IRREGULIER	vecteur + facteur d'etiquetage, sous-classes de tailles POSSIBLEMENT DIFFERENTES (« ragged array »)
ORDONNE	<code>ordered()</code> -- identique a <code>factor()</code> SAUF * l'affichage montre l'ordre des niveaux * les CONTRASTES engendres pour les modeles lineaires different
PIEGES	<code>x[facteur] = x[as.integer(facteur)]</code> les NIVEAUX NE SERVENT PAS <code>sd()</code> N'EST PAS l'erreur type

Le problème posé. « *A factor is a vector object used to specify a discrete classification (grouping) of the components of other vectors of the same length.* » (§4.1) Un facteur ne porte pas les données : il porte l'**appartenance** des données à des groupes. C'est un objet **relationnel** — il n'a de sens que rapporté à un autre vecteur.

INTUITION — POURQUOI CE CHAPITRE PARAÎT MINEUR ET NE L'EST PAS.

« While the **"real" application of factors is with model formulae** (see §11.1.1 Contrasts), we here look at a specific example. » (§4.1) Le cours prévient : ce qu'on voit ici — `tapply()` et des moyennes par groupe — n'est **pas** la raison d'être des facteurs. Leur vraie fonction est de dire à `lm()` et à `aov()` « *cette variable est catégorielle, engendre-lui des indicatrices* » (fiche 313). Le facteur est **le pont entre les données et les modèles**.

● Concept 1 — Créer un facteur, et ce qu'il devient

DÉFINITION (R LANGUAGE DEFINITION §2.3.1).

« Factors are used to describe items that can have **a finite number of values** (gender, social class, etc.). **A factor has a `levels` attribute and class "factor"** . Optionally, it may also contain a `contrasts` attribute which controls **the parametrisation used when the factor is used in a modeling function.** »

L'exemple qui court dans tout le chapitre (§4.1). « *Suppose we have a sample of **30 tax accountants from all the states and territories of Australia** and their individual **state of origin** is specified by a character vector of state mnemonics.* »

```
state <- c("tas", "sa", "qld", "nsw", "nsw", "nt", "wa", "wa",  
          "qld", "vic", "nsw", "vic", "qld", "qld", "sa", "tas",  
          "sa", "nt", "wa", "vic", "qld", "nsw", "nsw", "wa",  
          "sa", "act", "nsw", "vic", "vic", "act")
```

« Notice that in the case of **a character vector**, **"sorted"** means **sorted in alphabetical order**. » — la phrase qui explique l'ordre des niveaux au concept 2.

```
statef <- factor(state)
```

CE QUE FAIT PRINT() (§4.1).

« The `print()` function handles factors **slightly differently from other objects** » :

```
[1] tas sa qld nsw nsw nt wa wa qld vic nsw vic qld qld sa  
[16] tas sa nt wa vic qld nsw nsw wa sa act nsw vic vic act  
Levels: act nsw nt qld sa tas vic wa
```

Trois différences visibles avec un vecteur de caractères : pas de guillemets ; une ligne **Levels:** ; et cette ligne montre **tous** les niveaux, y compris ceux qui n'apparaîtraient pas dans les données affichées.

« To find out the levels of a factor **the function** `levels()` **can be used**. »

```
levels(statef)  
# [1] "act" "nsw" "nt" "qld" "sa" "tas" "vic" "wa"
```

(Note 1 du §4.1) « Readers should note that **there are eight states and territories in Australia** » — l'Australian Capital Territory, la Nouvelle-Galles du Sud, le Territoire du Nord, le Queensland, l'Australie-Méridionale, la Tasmanie, le Victoria et l'Australie-Occidentale. Le facteur a donc **exactement 8 niveaux pour 30 observations** : c'est bien une classification, pas une identification.

1.1 Sous le capot

CE QUE LE MANUEL DIT — ET CE QU'IL INTERDIT D'EN FAIRE (§2.3.1).

« Factors are **currently implemented** using **an integer array to specify the actual levels** and **a second array of names that are mapped to the integers**. **Rather unfortunately users often make use of the implementation in order to make some calculations easier**. **This, however, is an implementation issue and is not guaranteed to hold in all implementations of R.** »

Un facteur **est donc**, aujourd'hui, un vecteur d'entiers muni de `levels` et de `class`. On peut le vérifier :

```
unclass(statef) # montre les codes entiers et l'attribut levels
```

Mais le manuel demande explicitement de ne pas s'appuyer là-dessus. La distinction est importante :

`unclass()` sert à **comprendre** (fiche 303), pas à **calculer**.

Le piège qui découle directement de cette implémentation (*R Language Definition* §3.4.1, fiche 302). *« **Factor**. The result [de `x[i]` avec `i` facteur] is **identical to** `x[as.integer(i)]`. **The factor levels are never used**. If so desired, use `x[as.character(i)]` »*.

Indexer par un facteur indexe donc **par des positions**, que rien à l'écran ne laisse deviner — puisque le facteur **s'affiche avec ses étiquettes**. Le résultat a la bonne longueur et il est faux.

● Concept 2 — L'ordre des niveaux

Règle (§4.3). « *The levels of factors are stored in alphabetical order, or in the order they were specified to* `factor` *if they were specified explicitly.* »

C'est une règle en deux temps, et la seconde moitié est ce qui sauve :

```
# ordre par défaut : alphabétique
levels(factor(c("moyen", "fort", "faible")))

# "faible" "fort" "moyen"      <- alphabétique, donc DENUE DE SENS ici

# ordre impose
levels(factor(c("moyen", "fort", "faible"),
              levels = c("faible", "moyen", "fort")))

# "faible" "moyen" "fort"      <- l'ordre voulu
```

Pourquoi cela compte au-delà de l'esthétique. L'ordre des niveaux détermine : l'ordre des colonnes d'un `table()`, l'ordre des boîtes d'un `boxplot`, l'ordre des barres d'un graphique — et surtout **quel niveau sert de référence** dans un modèle linéaire (fiche 313). Le premier niveau est celui auquel tous les autres sont comparés. Le laisser à l'alphabet, c'est laisser l'alphabet choisir la modalité de référence.

● Concept 3 — `tapply()` : appliquer une fonction par groupe

DÉFINITION (§4.2).

« The function `tapply()` is used to **apply a function** — here `mean()` — **to each group of components of the first argument** — here `incomes` — **defined by the levels of the second component** — here `statef` — **as if they were separate vector structures. The result is a structure of the same length as the `levels` attribute of the factor** containing the results. »

Les données de revenus (§4.2), « in suitably large units of money » :

```
incomes <- c(60, 49, 40, 61, 64, 60, 59, 54, 62, 69, 70, 42, 56,
            61, 61, 61, 58, 51, 48, 65, 49, 49, 41, 48, 52, 46,
            59, 46, 58, 43)

incmeans <- tapply(incomes, statef, mean)
```

« giving a means vector with the components **labelled by the levels** » :

	act	nsw	nt	qld	sa	tas	vic	wa
moyenne	44,500	57,333	55,500	53,600	55,000	60,500	56,000	52,250

EN CLAIR — LE MÉCANISME, TEL QUE LE COURS LE DÉCRIT (§4.2).

« *The values in the vector are collected into groups corresponding to the distinct entries in the factor. The function is then applied to each of these groups individually. The value is a vector of function results, labelled by the levels attribute of the factor.* »

Trois temps : **regrouper, appliquer, étiqueter**. C'est le patron de toute l'analyse par sous-groupes en R.

Le deuxième argument n'a pas besoin d'être un facteur (note 2, §4.2) : *« Note that `tapply()` also works in this case when its second argument is not a factor, e.g. `tapply(incomes, state)`, and this is true for quite a few other functions, since arguments are coerced to factors when necessary (using `as.factor()`). »* Commode — et dangereux, puisque la coercion silencieuse applique l'**ordre alphabétique** du concept 2.

« The function `tapply()` can also be used to handle more complicated indexing of a vector by multiple categories. For example, we might wish to *split the tax accountants by both state and sex*. »

Énoncé. Vérifier, sans `tapply()`, que la moyenne de l'état `qld` vaut bien **53,600**.

Étape 1 — repérer les positions. Dans `state`, `"qld"` occupe les positions **3, 9, 13, 14, 21**.

Étape 2 — extraire les revenus correspondants. C'est exactement une indexation logique (fiche 302) :

```
incomes[state == "qld"]
# 40 62 56 61 49
```

Étape 3 — sommer. **40 + 62 + 56 + 61 + 49 = 268**.

Étape 4 — diviser par l'effectif. **268/5 = 53,6**. conforme au tableau du cours.

Étape 5 — comprendre ce que `tapply()` a fait de plus. Rien de mystérieux : il a répété les étapes 2 à 4 **pour chacun des huit niveaux**, puis a **étiqueté** le résultat. La formulation du cours — « *as if they were separate vector structures* » — est littérale.

Étape 6 — le point qui compte. La boucle sur les niveaux est **écrite par** `tapply()`, comme la boucle sur les éléments est écrite par la vectorisation (fiche 301). Le patron « regrouper, appliquer, étiqueter » remplace ici la boucle `for` imbriquée que l'on écrirait dans un autre langage.

3.1 L'erreur type — et le piège `sd()`

MÉTHODE (§4.2).

« Suppose further we needed to calculate **the standard errors of the state income means**. To do this we need to **write an R function to calculate the standard error for any given vector**. Since there is a builtin function `var()` to calculate the sample variance, such a function is **a very simple one liner** » :

```
stdError <- function(x) sqrt(var(x)/length(x))
```

$$se(\bar{x}) = \sqrt{\frac{\text{var}(x)}{n}} = \frac{s}{\sqrt{n}}$$

La mise en garde du cours, en une parenthèse. « *Note that R's builtin function `sd()` is something different.* » (§4.2)

`sd(x)` est l'écart-type de l'échantillon, soit $s = \sqrt{\text{var}(x)}$; l'erreur type de la moyenne est $s/\sqrt{n}$. Les deux diffèrent d'un facteur $\sqrt{n}$ — sur `nsw` ($n = 6$), d'un facteur **2,449**. Confondre les deux, c'est publier un intervalle de confiance **deux fois et demie trop large**.

```
incster <- tapply(incomes, statef, stdError)
```

	act	nsw	nt	qld	sa	tas	vic	wa
erreur type	1,5	4,3102	4,5	4,1061	2,7386	0,5	5,244	2,6575

CONTRÔLE — VÉRIFIER UNE VALEUR DU TABLEAU.

Pour `act`, les deux revenus sont **46** et **43**, de moyenne **44,5**. La variance d'échantillon vaut

$$\text{var} = \frac{(46 - 44,5)^2 + (43 - 44,5)^2}{2 - 1} = \frac{2,25 + 2,25}{1} = 4,5$$

et l'erreur type $\sqrt{4,5/2} = \sqrt{2,25} = 1,5$. Même vérification pour `tas` (60 et 61) : **var** = 0,5, $\sqrt{0,5/2} = 0,5$.

Énoncé (§4.2, textuel). « *As an exercise you may care to find the usual 95 % confidence limits for the state mean incomes. To do this you could use `tapply()` once more with the `length()` function to find the sample sizes, and the `qt()` function to find the percentage points of the appropriate t-distributions. (You could also investigate R's facilities for t-tests.)* »

Le cours ne donne pas de corrigé. Celui qui suit est **une correction pédagogique** construite à partir des indications qu'il fournit (`tapply` , `length` , `qt`) et de la théorie standard de l'intervalle de Student. Les valeurs numériques sont recalculées à partir des données du chapitre.

Étape 1 — les effectifs, comme le cours le suggère.

```
n <- tapply(incomes, statef, length)
```

	act	nsw	nt	qld	sa	tas	vic	wa
effectif	2	6	2	5	4	2	5	4

Contrôle : $2 + 6 + 2 + 5 + 4 + 2 + 5 + 4 = 30$, l'effectif total.

Étape 2 — la formule. L'intervalle de Student à $1 - \alpha$ pour une moyenne, l'écart-type étant estimé, est

$$\bar{x} \pm t_{1-\alpha/2, n-1} \cdot \frac{s}{\sqrt{n}}$$

où $s/\sqrt{n}$ est **exactement** ce que calcule `stdError` de l'étape précédente.

Étape 3 — les points de pourcentage, avec `qt()` comme indiqué. Le degré de liberté est $n - 1$, **différent pour chaque état** — c'est le point délicat de l'exercice.

```
t <- qt(0.975, df = n - 1)
```

effectif n	2	4	5	6
$\nu = n - 1$	1	3	4	5
$t_{0,975,\nu}$	12,706	3,182	2,776	2,571

Étape 4 — assembler.

```
bas <- incmeans - t * incster  
haut <- incmeans + t * incster
```

Étape 5 — deux résultats à lire attentivement.

Pour `nsw` ($n = 6$) : $57,333 \pm 2,571 \times 4,3102 = 57,333 \pm 11,08$, soit [46,25 ; 68,41].

Pour `act` ($n = 2$) : $44,5 \pm 12,706 \times 1,5 = 44,5 \pm 19,06$, soit [25,44 ; 63,56].

Étape 6 — interpréter. L'erreur type de `act` (1,5) est **la plus petite du tableau**, et son intervalle est pourtant **le plus large**. La raison est le facteur t : avec $\nu = 1$ degré de liberté, $t_{0,975}$ vaut **12,706** au lieu de 2,571. **Deux observations ne suffisent pas** — l'incertitude sur s lui-même écrase tout.

Étape 7 — la leçon de méthode. Une petite erreur type n'est pas un signe de précision si l'effectif est minuscule. `tapply(incomes, statef, length)` **doit toujours accompagner** `tapply(incomes, statef, mean)` : un groupe de taille 2 dans un tableau de moyennes est une valeur qu'on ne peut pas commenter.

Étape 8 — le renvoi du cours. « You could also investigate R's facilities for t-tests » — `t.test()`, qui produit directement l'intervalle. Voir fiche 312.

● Concept 4 — Le tableau irrégulier (*ragged array*)

DÉFINITION (§4.2).

« *The combination of a vector and a labelling factor is an example of what is sometimes called a ragged array, since the subclass sizes are possibly irregular. When the subclass sizes are all the same the indexing may be done implicitly and much more efficiently, as we see in the next section.* »

C'est le concept-charnière du chapitre, et il explique **pourquoi les facteurs existent** :

Structure	Tailles des sous-classes	Comment on indexe
Vecteur + facteur (tableau irrégulier)	quelconques	par les niveaux — <code>tapply()</code>
Tableau (<code>array</code>)	toutes égales	implicitement , par les dimensions — fiche 305

Un tableau à deux dimensions **impose** que chaque groupe ait le même effectif : c'est ce qui permet le calcul d'adresse direct (`dim` , fiche 303) et donc l'efficacité. Un facteur **lève cette contrainte**, au prix d'une indexation explicite. **Le chapitre 5 est la suite naturelle de cette phrase.**

COMPLÉMENT (R LANGUAGE DEFINITION §2.3.2).

Le data frame reprend la même idée : « *A data frame **can contain a list** that is the same length as the other components. The list can contain elements of **differing lengths**, thereby **providing a data structure for ragged arrays**. However, as of this writing such arrays are not generally handled correctly.* » — la mise en garde vaut d'être connue avant d'essayer.

● Concept 5 — Facteurs ordonnés

DÉFINITION (§4.3).

« *Sometimes **the levels will have a natural ordering that we want to record and want our statistical analysis to make use of**. The `ordered()` function creates such ordered factors but is otherwise identical to `factor`.* »

Règle — les deux seules différences (§4.3). « *For most purposes the only difference between ordered and unordered factors is that the former are printed showing the ordering of the levels, but the contrasts generated for them in fitting linear models are different.* »

	<code>factor()</code>	<code>ordered()</code>
Création	identique	identique
Affichage	Levels: a b c	Levels: a < b < c
Contrastes en modèle linéaire	traitement (indicatrices)	différents — polynomiaux
Classe (<i>R Language Definition</i> §2.3.1)	"factor"	<code>c("ordered", "factor")</code>

Ne pas se laisser tromper par « the only difference ». La seconde différence est **considérable**. Les contrastes déterminent **la paramétrisation** du modèle : avec un facteur non ordonné, les coefficients comparent chaque niveau **au niveau de référence** ; avec un facteur ordonné, ils décrivent une **tendance** (linéaire, quadratique...) le long des niveaux. Ce ne sont pas les mêmes coefficients, ni les mêmes hypothèses. Voir fiche 313 (§11.1.1 « Contrasts »).

« A factor may be **purely nominal** or may have **ordered categories**. In the latter case, **it should be defined as such** » (*R Language Definition* §2.3.1) — le manuel dit **doit**, pas *peut*.

Comment reconnaître le type d'exercice

Le signal dans l'énoncé	Ce qu'il faut mobiliser
« une variable catégorielle », « des groupes »	un facteur — <code>factor()</code>
« la moyenne par état », « par catégorie »	<code>tapply(donnees, facteur, fonction)</code>
« les effectifs par groupe »	<code>tapply(..., length)</code> — ou <code>table()</code>
« mes barres sont dans le mauvais ordre »	les niveaux sont alphabétiques → argument <code>levels =</code>
« quel niveau sert de référence ? »	le premier — donc celui que l'alphabet a choisi
« faible / moyen / fort »	des catégories ordonnées → <code>ordered()</code>
« les coefficients de mon modèle sont bizarres »	facteur ordonné → contrastes polynomiaux
« l'erreur type de la moyenne »	$s/\sqrt{n}$ — <code>sqrt(var(x)/length(x))</code> , pas <code>sd()</code>
« un intervalle de confiance par groupe »	<code>tapply(..., length) + qt(0.975, n-1)</code>
« des groupes de tailles inégales »	un tableau irrégulier — vecteur + facteur
« des groupes de tailles égales »	un tableau (<code>array</code>) — indexation implicite, fiche 305
« croiser deux variables catégorielles »	<code>tapply()</code> accepte plusieurs facteurs
« mon sous-ensemble est faux et de bonne longueur »	indexation par un facteur → <code>as.character()</code>

Comment résoudre ce type d'exercice

Protocole « analyser par groupes » — 5 étapes.

1. **Construire le facteur** — et **fixer l'ordre des niveaux** si l'alphabet ne convient pas.
2. `tapply(x, f, length)` d'abord : connaître **les effectifs** avant de commenter les moyennes.
3. `tapply(x, f, mean)` , ou toute autre statistique.
4. Pour la dispersion, écrire **la fonction dont on a besoin** — le cours montre qu'un one-liner suffit — et se souvenir que `sd()` **n'est pas l'erreur type**.
5. **Vérifier que le résultat a autant d'éléments que de niveaux** : « *a structure of the same length as the `levels` attribute* ».

Protocole « choisir entre `factor` et `ordered` » — 3 étapes.

1. Les catégories ont-elles **un ordre naturel** ? Non → `factor()` .
2. Oui, et cet ordre doit-il **entrer dans l'analyse** ? Oui → `ordered()` , en donnant `levels =` **dans le bon ordre**.
3. Vérifier l'affichage : `Levels: a < b < c` confirme l'ordre enregistré.

Protocole « intervalle de confiance par groupe » — 4 étapes.

1. `n <- tapply(x, f, length)` — **les degrés de liberté diffèrent d'un groupe à l'autre**.
2. `se <- tapply(x, f, function(v) sqrt(var(v)/length(v)))` .
3. `t <- qt(0.975, df = n - 1)` — **vectorisé sur** `n` (fiche 301).
4. `moyenne ± t * se` , puis **relire les groupes de petit effectif** : $t_{0,975,1} = 12,7$.

● Common mistakes

Erreur	Correction
Laisser l'ordre alphabétique des niveaux	« or in the order they were specified to factor » → argument <code>levels =</code>
Croire que l'ordre des niveaux n'est qu'esthétique	il fixe la modalité de référence d'un modèle
Utiliser <code>sd()</code> comme erreur type	l'erreur type est $s/\sqrt{n}$
Commenter des moyennes sans les effectifs	<code>tapply(..., length)</code> d'abord
Croire une petite erreur type synonyme de précision	avec $n = 2$, $t_{0,975} = 12,7$
Indexer par un facteur	c'est <code>x[as.integer(f)]</code> → <code>x[as.character(f)]</code>
Calculer sur les codes entiers d'un facteur	« an implementation issue ... not guaranteed »
Convertir un facteur numérique par <code>as.numeric()</code>	cela rend les codes , pas les étiquettes
Croire <code>ordered()</code> purement cosmétique	il change les contrastes du modèle
Oublier <code>levels =</code> avec <code>ordered()</code>	sans lui, l'ordre reste alphabétique
Passer un vecteur de caractères à <code>tapply()</code> sans y penser	il est coercé par <code>as.factor()</code> — donc ordre alphabétique
Attendre d'un tableau irrégulier l'efficacité d'un <code>array</code>	« when the subclass sizes are all the same the indexing may be done implicitly and much more efficiently »
Confondre le nombre de niveaux et le nombre d'observations	8 niveaux, 30 observations

Ultimate Review

Définition. « A *factor* is a **vector object used to specify a discrete classification (grouping) of the components of other vectors of the same length**. » Il porte **l'appartenance**, pas les données.

Sa vraie fonction. « *The "real" application of factors is with model formulae* » — les contrastes, fiche 313. `tapply()` n'en est qu'une illustration.

Structure. Attribut `levels` + classe `"factor"` (+ `contrasts` en option). Implémenté par un **tableau d'entiers** et un tableau de noms — « *an implementation issue and is not guaranteed* ».

Création et affichage. `factor(x)` · `levels(f)` pour lire les niveaux · `print()` traite les facteurs **différemment** : pas de guillemets, et une ligne `Levels:` listant **tous** les niveaux.

L'ordre des niveaux. « *stored in alphabetical order, or in the order they were specified to factor if they were specified explicitly* ». Pour les caractères, « *"sorted" means sorted in alphabetical order* ». L'ordre commande l'affichage, les tableaux, les graphiques — **et la modalité de référence** d'un modèle.

`tapply()` . « *apply a function ... to each group of components of the first argument, defined by the levels of the second, as if they were separate vector structures* ». Résultat : « *a structure of the same length as the `levels` attribute* », **étiqueté par les niveaux**. Mécanisme en trois temps : **regrouper, appliquer, étiqueter**. Le deuxième argument **n'a pas besoin d'être un facteur** — il est coercé par `as.factor()` . Accepte **plusieurs catégories**.

Erreur type. `stdError <- function(x) sqrt(var(x)/length(x))` — « *R's builtin function `sd()` is something different* ». Écart d'un facteur $\sqrt{n}$.

Les chiffres du chapitre. 30 comptables, 8 états et territoires · moyennes : act **44,500** · nsw **57,333** · nt **55,500** · qld **53,600** · sa **55,000** · tas **60,500** · vic **56,000** · wa **52,250** · erreurs types : **1,5** · 4,3102 · **4,5** · 4,1061 · 2,7386 · **0,5** · 5,244 · 2,6575 · effectifs : 2, 6, 2, 5, 4, 2, 5, 4.

Tableau irrégulier. « *The combination of a vector and a labelling factor ... since the subclass sizes are possibly irregular* ». Quand les sous-classes sont **de même taille**, « *the indexing may be done implicitly and much more efficiently* » — c'est le tableau (fiche 305).

Facteurs ordonnés. `ordered()` — « *otherwise identical to `factor`* ». Deux différences : **l'affichage montre l'ordre** (`a < b < c`) et **les contrastes engendrés en modèle linéaire diffèrent**. Classe `c("ordered", "factor")` . « *In the latter case, it should be defined as such* ».

Active Recall

« *A factor is a vector object used to specify a discrete classification (grouping) of the components of other vectors of the same length.* » (§4.1)

Deux points dans cette définition : il **classe** (il ne mesure pas), et il classe **les composantes d'autres vecteurs de même longueur** — c'est un objet **relationnel**, sans intérêt seul.

Sa vraie fonction : « *While the "real" application of factors is with model formulae (see Contrasts), we here look at a specific example.* » Le chapitre 4 est une illustration ; les facteurs existent pour dire aux fonctions de modélisation qu'une variable est catégorielle.

« *The levels of factors are stored in alphabetical order, or in the order they were specified to `factor` if they were specified explicitly.* » (§4.3)

Pourquoi cela compte : l'ordre des niveaux détermine l'ordre d'un `table()` , l'ordre des boîtes d'un `boxplot` , l'ordre des barres — et surtout **quel niveau sert de référence** dans un modèle linéaire. Le premier niveau est celui auquel les autres sont comparés.

Sur des catégories comme `faible/moyen/fort` , l'ordre alphabétique donne `faible`, `fort`, `moyen` — **dénué de sens**. La correction est l'argument `levels =` .

« *The function `tapply()` is used to **apply a function**, here `mean()` , **to each group of components of the first argument**, here `incomes` , **defined by the levels of the second component**, here `statef` , **as if they were separate vector structures. The result is a structure of the same length as the `levels` attribute of the factor** containing the results.* » (§4.2)

Et la description du mécanisme : « *The values in the vector are collected into groups corresponding to the distinct entries in the factor. The function is then applied to each of these groups individually. The value is a vector of function results, labelled by the levels attribute of the factor.* »

Trois temps : **regrouper, appliquer, étiqueter**.

Non. Note 2 du §4.2 : *« Note that `tapply()` **also works in this case when its second argument is not a factor**, e.g. `tapply(incomes, state)` , and **this is true for quite a few other functions, since arguments are coerced to factors when necessary (using `as.factor()`).** »*

Mais c'est un confort risqué : la coercition silencieuse crée un facteur **aux niveaux alphabétiques**. Si l'ordre importe — et il importe dès qu'un graphique ou un modèle suit — il faut construire le facteur **explicitement**, avec `levels = .`

Parce que la fonction demandée est **l'erreur type de la moyenne** :

```
stdError <- function(x) sqrt(var(x)/length(x))
```

$$se(\bar{x}) = \frac{s}{\sqrt{n}} \quad \text{alors que} \quad sd(x) = s$$

Les deux diffèrent d'un facteur $\sqrt{n}$. Sur `nsw` ($n = 6$), `sd` vaudrait $4,3102 \times \sqrt{6} = 10,56$ contre une erreur type de **4,3102**. Confondre les deux produit un intervalle de confiance **2,45 fois trop large**.

C'est aussi pourquoi le cours fait **écrire** la fonction : elle n'existe pas en standard.

Les deux revenus de `act` sont **46** et **43** (positions 26 et 30), de moyenne **44,5**.

$$\text{var} = \frac{(46 - 44,5)^2 + (43 - 44,5)^2}{2 - 1} = \frac{2,25 + 2,25}{1} = 4,5$$

$$se = \sqrt{\frac{4,5}{2}} = \sqrt{2,25} = 1,5$$

conforme au tableau du cours. Même contrôle pour `tas` (60 et 61) : **var = 0,5** et **se = $\sqrt{0,25} = 0,5$** .

`act` a l'erreur type **la plus petite du tableau** (1,5) et l'intervalle **le plus large**.

La raison est le **degré de liberté**. L'intervalle est $\bar{x} \pm t_{0,975, n-1} \cdot se$, et pour $n = 2$:

$$t_{0,975, 1} = \mathbf{12,706} \quad \text{contre} \quad t_{0,975, 5} = 2,571$$

Donc $44,5 \pm 12,706 \times 1,5 = 44,5 \pm 19,06$, soit $[25,44; 63,56]$ — bien plus large que $[46,25; 68,41]$ pour `nsw`.

La leçon : deux observations ne suffisent pas. `tapply(x, f, length)` **doit toujours accompagner** `tapply(x, f, mean)`.

(Correction pédagogique : le cours pose cet exercice au §4.2 sans en donner la solution.)

« *The combination of a vector and a labelling factor is an example of what is sometimes called a ragged array, since the subclass sizes are possibly irregular. When the subclass sizes are all the same the indexing may be done implicitly and much more efficiently, as we see in the next section.* » (§4.2)

	Tailles des sous-classes	Indexation
Vecteur + facteur	quelconques	explicite , par les niveaux
Tableau (<code>array</code>)	toutes égales	implicite , par les dimensions

Un `array` **impose** l'égalité des effectifs, ce qui permet un calcul d'adresse direct via `dim` (fiche 303) — d'où l'efficacité. Le facteur **lève cette contrainte**, au prix d'une indexation explicite.

« *The `ordered()` function creates such ordered factors but is otherwise identical to `factor`. For most purposes the only difference between ordered and unordered factors is that the former are printed showing the ordering of the levels, but the contrasts generated for them in fitting linear models are different.* » (§4.3)

1. **L'affichage** : `Levels: a < b < c` au lieu de `Levels: a b c`.
2. **Les contrastes** en modèle linéaire — ce qui est **tout sauf mineur** : avec un facteur non ordonné, les coefficients comparent chaque niveau au **niveau de référence** ; avec un facteur ordonné, ils décrivent une **tendance** le long des niveaux.

Et la classe diffère : `c("ordered", "factor")` (*R Language Definition* §2.3.1), qui ajoute « *it should be defined as such* » quand l'ordre existe.

*« **Factor**. The result is **identical to** `x[as.integer(i)]` . **The factor levels are never used**. If so desired, use `x[as.character(i)]` or a similar construction. »* (*R Language Definition* §3.4.1)

C'est le piège le plus vicieux du chapitre : le facteur **s'affiche avec ses étiquettes** mais **vaut ses codes entiers**. On indexe donc **par des positions 1, 2, 3...** que rien à l'écran ne laisse deviner. Le résultat a **la bonne longueur** et il est **faux**.

Le remède est `as.character()` .

Non, et le manuel est explicite : « *Factors are **currently implemented** using an integer array to specify the actual levels and a second array of names that are mapped to the integers. **Rather unfortunately users often make use of the implementation in order to make some calculations easier. This, however, is an implementation issue and is not guaranteed to hold in all implementations of R.*** » (*R Language Definition* §2.3.1)

`unclass(f)` **montre** les codes — c'est l'outil pédagogique du §3.4 d'*An Introduction to R* — mais montrer n'est pas **calculer avec**.

Le corollaire pratique le plus coûteux : sur un facteur dont les étiquettes sont des nombres, `as.numeric(f)` **rend les codes, pas les valeurs**. Il faut `as.numeric(as.character(f))` .

Huit — `act` , `nsw` , `nt` , `qld` , `sa` , `tas` , `vic` , `wa` — pour **30** observations.

Le cours ajoute en note 1 : « *Readers should note that **there are eight states and territories in Australia*** ». La précision sert à confirmer que **tous** les niveaux possibles sont représentés : le facteur n'a pas de niveau vide, et l'échantillon couvre l'ensemble du territoire.

Elle rappelle surtout la distinction **niveaux / observations** : un facteur classe **beaucoup** d'observations dans **peu** de catégories. Si les deux nombres étaient proches, ce ne serait pas une classification mais une identification — et `tapply()` n'aurait aucun sens.

Flashcards

Question	Réponse
Qu'est-ce qu'un facteur ?	Un vecteur qui classe les composantes d'autres vecteurs de même longueur
Sa « vraie » application selon le cours ?	Les formules de modèle — les contrastes
Ses deux attributs obligatoires ?	<code>levels</code> et <code>class = "factor"</code>
Un attribut optionnel ?	<code>contrasts</code>
Comment le créer ?	<code>factor(x)</code>
Comment lire ses niveaux ?	<code>levels(f)</code>
Comment <code>print()</code> traite-t-il un facteur ?	Sans guillemets , avec une ligne <code>Levels:</code>
L'ordre par défaut des niveaux ?	Alphabétique
Comment l'imposer ?	L'argument <code>levels =</code> de <code>factor()</code>
Pourquoi cet ordre compte-t-il ?	Il fixe la modalité de référence d'un modèle
Pour un vecteur de caractères, « trié » signifie ?	Par ordre alphabétique
Combien de niveaux dans l'exemple ?	8 états et territoires
Pour combien d'observations ?	30 comptables
À quoi sert <code>tapply()</code> ?	Appliquer une fonction à chaque groupe défini par les niveaux
Ses trois arguments dans l'exemple ?	<code>incomes</code> , <code>statef</code> , <code>mean</code>
Longueur du résultat ?	Celle de l'attribut <code>levels</code>
Comment est-il étiqueté ?	Par les niveaux
Les trois temps du mécanisme ?	Regrouper, appliquer, étiqueter
Le 2 ^e argument doit-il être un facteur ?	Non — coercé par <code>as.factor()</code>
Quel risque cela crée-t-il ?	Des niveaux en ordre alphabétique

Question	Réponse
<code>tapply()</code> accepte-t-il plusieurs catégories ?	Oui
La formule de l'erreur type ?	<code>sqrt(var(x)/length(x))</code>
Que vaut-elle en notation mathématique ?	$s/\sqrt{n}$
<code>sd()</code> est-elle l'erreur type ?	Non — c'est s
Facteur entre les deux ?	$\sqrt{n}$
Moyenne de <code>qld</code> ?	53,600
Moyenne de <code>act</code> ?	44,500
Erreur type de <code>act</code> ?	1,5
Erreur type de <code>tas</code> ?	0,5
Erreur type la plus grande ?	5,244 (<code>vic</code>)
Effectif de <code>act</code> ?	2
Effectif de <code>nsw</code> ?	6
Quelle fonction pour les effectifs ?	<code>tapply(x, f, length)</code>
Quelle fonction pour les points de Student ?	<code>qt()</code>
Degré de liberté d'un groupe de taille <code>n</code> ?	<code>n - 1</code>
$t_{0,975}$ pour $\nu = 1$?	12,706
$t_{0,975}$ pour $\nu = 5$?	2,571
Pourquoi <code>act</code> a-t-il l'intervalle le plus large ?	Un seul degré de liberté
La formule de l'intervalle ?	$\bar{x} \pm t_{1-\alpha/2, n-1} \cdot s/\sqrt{n}$
Autre piste suggérée par le cours ?	Les facilités de test t de R
Qu'est-ce qu'un tableau irrégulier ?	Un vecteur + un facteur d'étiquetage
Pourquoi « irrégulier » ?	Les tailles des sous-classes peuvent différer

Question	Réponse
Que gagne-t-on si elles sont égales ?	Une indexation implicite et bien plus efficace
Quelle structure permet cela ?	Le tableau (<code>array</code>) — fiche 305
Un data frame peut-il porter un tableau irrégulier ?	Oui, via une liste — mais « <i>not generally handled correctly</i> »
Comment créer un facteur ordonné ?	<code>ordered()</code>
En quoi diffère-t-il de <code>factor()</code> ?	« <i>otherwise identical</i> »
Sa première différence ?	L'affichage montre l'ordre : <code>a < b < c</code>
Sa seconde différence ?	Les contrastes en modèle linéaire
Sa classe ?	<code>c("ordered", "factor")</code>
Le manuel dit-il « peut » ou « doit » ?	« <i>it should be defined as such</i> »
Comment est implémenté un facteur ?	Un tableau d'entiers + un tableau de noms
Peut-on s'appuyer dessus ?	Non — « <i>an implementation issue ... not guaranteed</i> »
Que rend <code>x[f]</code> avec <code>f</code> facteur ?	<code>x[as.integer(f)]</code> — les niveaux ne servent pas
Comment indexer par les étiquettes ?	<code>x[as.character(f)]</code>
Que rend <code>as.numeric(f)</code> sur des étiquettes numériques ?	Les codes , pas les valeurs
La bonne conversion ?	<code>as.numeric(as.character(f))</code>
Comment voir un facteur « à nu » ?	<code>unclass(f)</code>